

CHAPTER 11

進階設計概念

Working

本章將為各位介紹：資源共用 (Resource Sharing)、Verilog 的編譯命令 (Compiler Directives)、易於調整的設計方式 (Scalable Design)，以及撰寫經濟實用的 HDL 程式碼之原則…等等的觀念，作一個簡要的說明。



11.1 資源共用 (Resource Sharing)

如果在某個選擇性的敘述，例如：if ... else ... 敘述 (二選一) 或 case 類 (多選一) 的敘述中，有相同功能的運算，則這些需要相同功能的運算在同一個時脈週期期間，最多只有一個相同運算會動作而已，因此只需使用一個運算處理電路；所以可以減少合成出來電路的元件數目，電路的面積也就因而降低了。

(一) 資源無法用的範例

例 1 有加法 (+) 電路，也有減法 (-)，運算不相同，因此無法共用。

```
always @(Select or a or b or c)
begin
    if (Select)
        z = a + b;
    else
        z = a - c;
end
```

(二) 資源可以共用的範例

例 2 因為運算相同，加法 (+) 電路可共用。

```
always @(Select or a or b or c)
begin
    if (Select)
        z = a + b;
    else
        z = a + c;
end
```

例 3 因為運算相同，加法(+) 電路可共用。

```
always @(Select or a or b or c)
begin
    case (Select)
        1'b0:
            begin
                z = a + b;
            end
        1'b1:
            begin
                z = a + c;
            end
    endcase
end
```

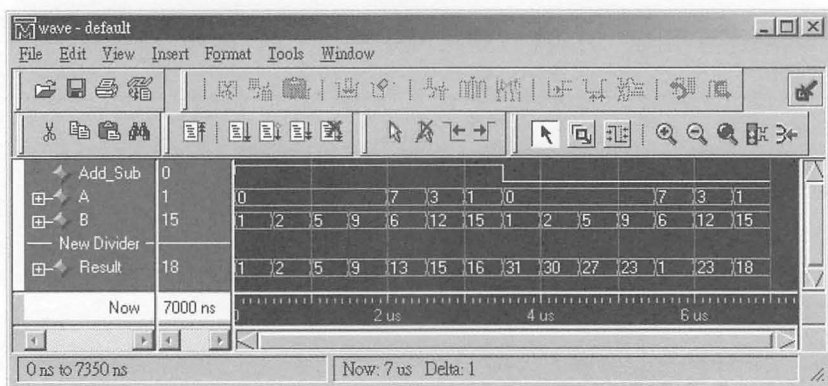
(三) 必須經過修改，資源才可以共用的範例

例 4 沒有「資源共用」的電路設計寫法 (Without Resource Sharing)

有加法(+) 電路，也有減法(-)，運算不相同，因此無法共用。

```
// Res_NShr.V: 沒有「資源共用」的電路設計寫法 (Without Resource Sharing)
// 因為會產生 1 個 4 位元的 Adder 及 1 個 4 位元的 Subtractor
// 所以產生的電路面積較大
module Without_Shareing_Add_Sub (Add_Sub, A, B, Result);
input      Add_Sub;
input      [3:0] A, B;
output     [4:0] Result;
reg        [4:0] Result;

always @(Add_Sub or A or B)
begin
    if (Add_Sub)
        Result = A + B;
    else
        Result = A - B;
    end
endmodule
```



▲ Res_NShr.V 的模擬結果

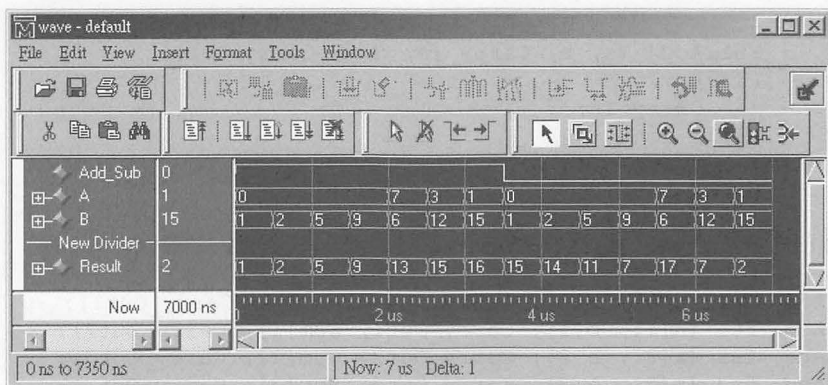
例 5 有「資源共用」的電路設計寫法 (With Resource Sharing)

將減法動作改用加法動作來處理，使得運算性質相同，因此加法 (+) 電路的部份變成可以共用。

```
// Res_NShr.V: 沒有「資源共用」的電路設計寫法 (Without Resource Sharing)
// 因為會產生 1 個 4 位元的 Adder 及 1 個 4 位元的 Subtractor
// 所以產生的電路面積較大
module With_Shareing_Add_Sub (Add_Sub, A, B, Result);
input      Add_Sub;
input  [3:0] A, B;
output  [4:0] Result;
reg      [4:0] Result;

always @(Add_Sub or A or B)
begin: Add_or_Sub // 命名為 Add_or_Sub 的 always 區塊
    // 可以在有命名的 always 區塊中，宣告區域性的變數
    reg [3:0] Temp; // 例: Temp 為一個區域性的變數

    if (Add_Sub)
        Temp = B;
    else
        Temp = ~B; // ~B = /B, 需要 4 個 Inverter
    Result = A + Temp + !Add_Sub; // 需要 1 個 4 位元的 Adder
end // (1) A + B + !1 = A + B,
    // (2) A - B + !0 = A + /B + 1
endmodule
```



▲ Res_WShr.V 的模擬結果



11.2 Verilog 的編譯命令 (Compiler Directives)

11.2.1 `include 編譯命令

`include 編譯命令的功能為：目前的 Verilog 電路描述檔可以包含另一個 Verilog 電路描述檔來一起編譯。

以下面的這個例子來說明：因為我們要設計的模組功能為一個具有 4 位元的漣波進位加法器 (4 Bits Ripple Carry Adder) 的 Verilog 電路描述檔 (FullAdd4.V)，因此我們可以引用已經設計好的一個具有 1 位元的全加器之 Verilog 電路描述檔 (FullAdd.V) 來一起編譯合成。

範例 4 位元的漣波進位加法器 (4 Bits Ripple Carry Adder)

【FullAdd4.V，4 位元漣波進位計數器】

```
// FullAdd4.V, 4 位元漣波進位計數器
`include "FullAdd.V" // 引用 FullAdd.V 檔中的所有可用模組
```

```
module    FullAdd4 (a, b, Carry_In, Sum, Carry_Out);
input     [3:0] a, b;
input     Carry_In;
output    [3:0] Sum;
output    Carry_Out;
wire      [3:0] Sum;
wire      Carry_Out;
// 注意：底下的 Carry_Out1, Carry_Out2, Carry_Out3 等變數可不宣告

    FullAdd fa0 (a[0], b[0], Carry_In, Sum[0], Carry_Out1);
    FullAdd fa1 (a[1], b[1], Carry_Out1, Sum[1], Carry_Out2);
    FullAdd fa2 (a[2], b[2], Carry_Out2, Sum[2], Carry_Out3);
    FullAdd fa3 (a[3], b[3], Carry_Out3, Sum[3], Carry_Out );
endmodule
```

【FullAdd.V，1 位元的全加器】

```
// FullAdd.V, 1 位元的全加器
module FullAdd (a, b, CarryIn, Sum, CarryOut);
input  a, b, CarryIn;
output Sum, CarryOut;
wire   Sum, CarryOut;

    assign {CarryOut, Sum} = a + b + CarryIn;
endmodule
```

11.2.2 `define 編譯命令

`define 編譯命令的功能為：類似 parameter 敘述。

我們可以使用 define 編譯命令來表示：有限狀態機器的內部狀態以及電路中會用到的常數值。

範例 `define 編譯命令、宣告方式及使用方法

```
reg      [1:0] state;

// 定義：一些在底下會用到的常數值
`define  RED_Light_Time 9 // 紅燈亮的時間倍值
// 定義：有限狀態機器的內部狀態，各個狀態的編碼
```



```

parameter RED_Light    = 2'b00, GREEN_Light = 2'b01,
          YELLOW_Light = 2'b10;
// ...
if (Reset) // 電路重置
    state = RED_Light; // 初始狀態設為: RED_Light (紅燈)
else
    case (state)
        RED_Light: // 目前狀態為: RED_Light (紅燈)
            if (Count_In == `RED_Light_Time)
                begin // 重新計數 Counter_16 設為: 1 (重新計數)
                    // ...
                end
            // ...
    end
// ...

```



11.3 易於調整的設計方式 (Scalable Design)

在 Verilog 描述電路中，可以下列二種方法來達到擴充 adder 這個具有易於調整性的模組。

- 使用 defparam 敘述去取代 (overriding) parameter 值。
- 使用 `# 敘述去取代 (overriding) parameter 值。

範例 具有易於調整性的加法器模組設計

【Scalable.V，建立一個具有易於調整性的 Adder】

```

// Scalable.V: 建立一個具有易於調整性的 Adder
module Adder (A, B, Carry_In, Sum, Carry_Out);
parameter size = 1; // 宣告一個名為 size 的 parameter, 且等於 1
input [size-1:0] A, B; // 'A' 和 'B' 也使用了名為 size 的這個 parameter
input Carry_In;
output [size-1:0] Sum; // 'Sum' 使用了名為 size 的這個 parameter
output Carry_Out;
wire [size-1:0] Sum;
// output [size-1:0] Sum; 和 output [1-1:0] Sum; 是相等的
wire Carry_Out;

assign {Carry_Out, Sum} = A + B + Carry_In;
endmodule

```

11.3.1 使用 defparam 敘述去取代 (overriding) parameter 值

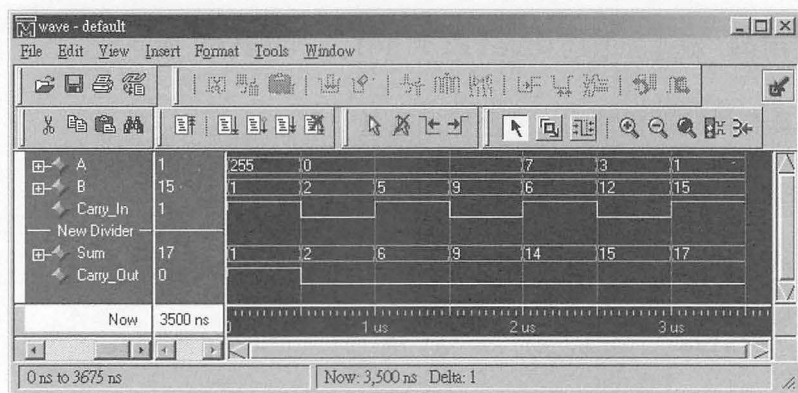
範例 Adder8.V，使用 defparam 敘述去 overriding parameter 值

```
// Adder8.V: 使用 defparam 敘述去 overriding parameter 值
`include "Scalable.V" // 引用 Scalable.V, 一個具有易於調整性的 Adder

module Adder8 (A, B, Carry_In, Sum, Carry_Out);
parameter our_size = 8; // 定義為 8 個位元
input      [our_size-1: 0] A, B;
input      Carry_In;
output     [our_size-1: 0] Sum;
output     Carry_Out;

    Adder    My_Adder8 (A, B, Carry_In, Sum, Carry_Out);
    /* defparam My_Adder8.size 將會改寫在 My_Adder8
        這個 Adder 別名中的 size 值 */
    defparam My_Adder8.size = 8; // 定義為 8 個位元

endmodule
```



▲ Adder8.V 的模擬結果

11.3.2 使用 '#' 敘述去取代 (overriding) parameter 值

範例 Adder16.V，使用 '#' 敘述去取代 (overriding) parameter 值

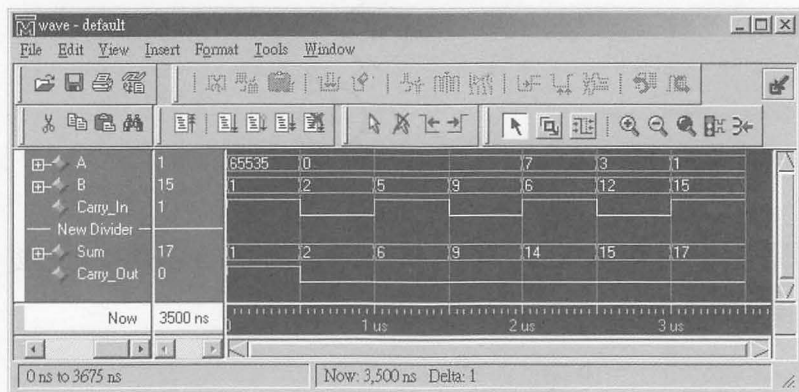

```
// Adder16.V: 使用 '#' 敘述去取代 (overriding) parameter 值
// 來達到擴充 Adder 這個具有易於調整性的模組

`include "Scalable.V" //引用 Scalable.V, 一個具有易於調整性的 Adder

module Adder16 (A, B, Carry_In, Sum, Carry_Out);
parameter our_size = 16; // 定義為 16 個位元
input      [our_size-1: 0] A, B;
input      Carry_In;
output     [our_size-1: 0] Sum;
output     Carry_Out;

/* our_size 將會改寫在 My_Adder16 這個 Adder 別名中的 size 值 */
Adder #(our_size) My_Adder16 (A, B, Carry_In, Sum, Carry_Out);

endmodule
```



▲ Adder16.V 的模擬結果



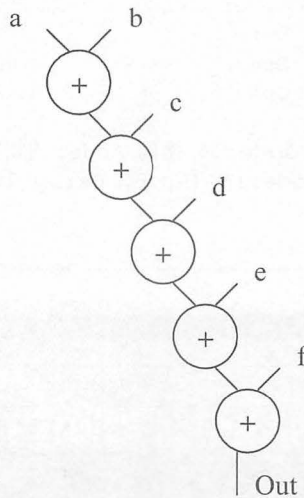
11.4 撰寫經濟實用的 HDL 程式碼之原則

11.4.1 使用括弧來描述複雜的電路結構

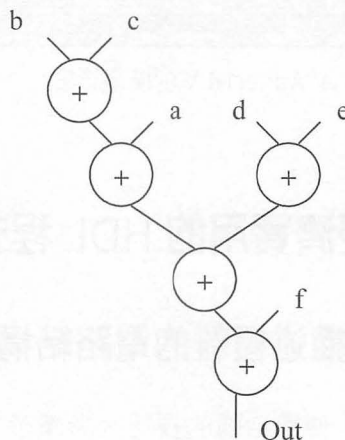
在底下的範例中：例 2 使用括弧的技巧，考慮了：訊號到達的時間 (Data

Arrival Time) 及訊號相依 (Data Dependency) 的關係，來得到功能相同；而且使用一樣多個加法電路，但因為加法動作的層數不同，所以例 2 合成出來的電路速度會比例 1 還要快。

例 1 $Out = a + b + c + d + e + f;$ // 5 層加法動作後，才會得到結果 Out



例 2 $Out = ((a + (b + c)) + (d + e)) + f;$ // 4 層加法動作後，就可以得到結果 Out



11.4.2 將「組合指定」敘述和「循序指定」敘述的部份描述區分開來

爲了減少合成出來電路中之暫存器(Register)的數目，將“組合指定”敘述(Combinational Assignment)和“循序指定”敘述(Sequential Assignment)的部份描述區分開來。

例 1 混合了“組合指定”敘述和“循序指定”敘述的部份描述

```
always @(posedge Clock)
begin
  if (Enable)
    Out = In;
  else
    Out = 1'bz;
end
```

例 2 將“組合指定”敘述和“循序指定”敘述的部份描述區分開來

```
always @(posedge Clock)
begin
  temp = In;
end

always @(Enable or temp)
begin
  if (Enable)
    Out = temp;
  else
    Out = 1'bz;
end
```

11.4.3 善用「常數」(Constant)

Verilog 的常數，大致有二種方式來表示：parameter 敘述及`define 編譯命令。

使用這二種方式來設計模組的優點，如下：

- 電路的模組易於令人接受與了解。
- 讓日後模組的修改及維護更為容易。

(一) parameter 敘述

例 1 parameter 敘述的使用範例

同樣是相加的處理，但它們所需的電路面積卻不盡相同。

像是： $a = size + 2$ ；其中 $size$ 已知為 8，所以就可以事先決定 a 的值必為 10，所以 $a = size + 2$ 經處理後就不是個需要加法的電路，純粹只是個固定等於 10 的值。

而在 $b = a + 1$ ；這個敘述中，只要把 a 的值再加上 1 後給 b ，所以這個敘述只需要加 1 器 (Incrementer) 即可完成加 1 的處理，並不需要一個加法器 (Adder) 才能運作。在電路面積的比較上：一個加 1 器 (Incrementer) 的電路面積遠小於一個加法器 (Adder) 的電路面積。

而在 $c = d + e$ ；這個敘述中，因為 d 及 e 為未知值 (一般為由輸入埠傳入的資料或是運算過程中產生的結果，要把 d 的值再加上 e 的值後給 c ，這個動作就需要加法器 (Adder) 才能完成。

```
parameter size = 8;
wire [3:0] a, b, c, d, e;

assign a = size + 2; // 直接給定常數值 (Constant)
assign b = a + 1;   // 加 1 器 (Increaser)
assign c = d + e;   // 加法器 (Adder)
```

例 2 parameter 敘述的使用範例

我們可以使用 parameter 敘述來定義：輸出入埠的寬度、向量 (Vectors) 表示法 (變數的位元數)、陣列 (Arrays) 表示法 (陣列的大小)，以及記憶體 (Memory) 表示法。

```
parameter our_size = 16; // 定義為 16 個位元
parameter Mem_1K = 16; // 定義一個名字為 Mem_1K 的參數

input      [our_size-1: 0] A, B; // 輸入埠的寬度
output     [our_size-1: 0] Sum; // 輸出埠的寬度

// 向量 (Vectors) 表示法
reg Mem1Bit [Mem_1K-1:0]; // 1024 個 1 Bits 的 Register

// 陣列 (Arrays) 表示法
reg [our_size-1:0] Word; // Word 為 1 個 Register, 且擁有 16 個 Bits

// 記憶體 (Memory) 表示法
reg [our_size-1:0] RegFile [Mem_1K:0];
// 1024 個 Register, 每個 Register 為 16 個 Bits
```

例 3 parameter 敘述的使用範例

我們可以使用 parameter 敘述來表示：有限狀態機器的內部狀態。

```
input      Clock, Reset;
reg        [1:0] state;

// 定義：有限狀態機器的內部狀態，各個狀態的編碼
parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin
        state = A; // 初始狀態設為 A
    end
end
```

(二) `define 編譯命令

範例 `define 編譯命令的使用範例

我們可以使用`define 編譯命令來表示：有限狀態機器的內部狀態以及電路中會用到的常數值。

```
reg    [1:0] state;

// 定義：一些在底下會用到的常數值
`define RED_Light_Time    9 // 紅燈亮的時間倍值
`define GREEN_Light_Time  6 // 綠燈亮的時間倍值
`define YELLOW_Light_Time 1 // 黃燈亮的時間倍值

// 定義：有限狀態機器的內部狀態，各個狀態的編碼
parameter RED_Light = 2'b00, GREEN_Light = 2'b01,
          YELLOW_Light = 2'b10;

always @(posedge Clock)
begin
    if (Reset) // 電路重置
    begin // 初始狀態設為：RED_Light (紅燈)
        state = RED_Light;
        RED    = 1'b1; // 紅燈亮
        GREEN  = 1'b0;
        YELLOW = 1'b0; // 重新計數 Counter_16 設為：1 (重新計數)
    end
    else
    begin
        case (state)
            RED_Light: // 目前狀態為：RED_Light (紅燈)
                begin // 紅燈：維持 RED_Light_Time 個時間單位
                    if (Count_In == `RED_Light_Time)
                    begin // 重新計數 Counter_16 設為：1 (重新計數)
                        end
                    .
                    . 略
                    .
                end
            endcase
        end
```


11.4.4 運算元的位用寬度 (Operator Bit-Width)

電路中沒有用到的部份儘量不要去使用它，就可以因此減少不必要的接線數目。

以底下這個例子來說，因為 temp 最大只會放 10，所以只需 4 位元就夠用了，Add_LT_10 = a + temp[3:0]；這行敘述原本需要由 8 個位元的加法電路，這種寫法就只需要少掉 4 個位元作進位處理的 8 個位元的加法電路。

範例 使用函數設計最大加 10 的處理

```
// Add_LT10.V：使用函數設計最大加 10 的處理
module Add_LT10 (a, b, c);
input  [7:0] a, b;
output [8:0] c;
wire   [8:0] c;

assign c = Add_LT_10 (a, b);

function [8:0] Add_LT_10;
input  [7:0] a, b;
reg    [7:0] temp;

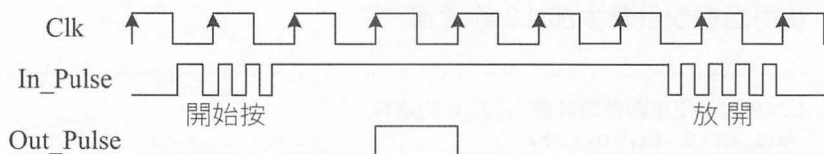
begin
    if (b < 10)
        temp = b;
    else
        temp = 10;    // 因為 temp 最大只會放 10，所以 4 位元就夠用了
        Add_LT_10 = a + temp[3:0]; // temp[3:0] 可存數值範圍：0 ~ 15.
        // 因為 temp 必小於等於 10，所以只要使用 temp 的 [3:0] 就可以了.
    end
endfunction

Endmodule
```



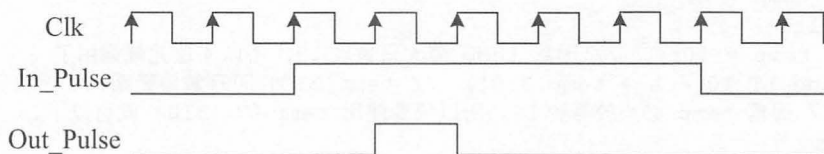
11.5 除彈跳電路 (DeBounce circuit) 與 單一脈波電路 (Mono pulse circuit)

除彈跳電路 (DeBounce circuit) 通常用於將您按下“按鈕開關”所產生的訊號，經由除彈跳電路確保對“按鈕開關”每按一次只會產生一個脈波寬度的邏輯‘1’輸出訊號；不過輸出的 pulse 訊號與輸入 pulse 訊號的 timing 關係至少會有 1 個 clock 的延遲。



先前是邏輯‘0’接著至少有連續 2 個時脈訊號都看到邏輯‘1’，那麼就輸出 1 個時脈訊號寬度的邏輯‘1’。

單一脈波電路 (Mono pulse circuit) 通常用於將數個連續的邏輯‘1’訊號，經由單一脈波電路之後產生一個脈波寬度的邏輯‘1’輸出訊號；不過輸出的 pulse 訊號與輸入 pulse 訊號的 timing 關係至少會有 1 個 clock 的延遲。



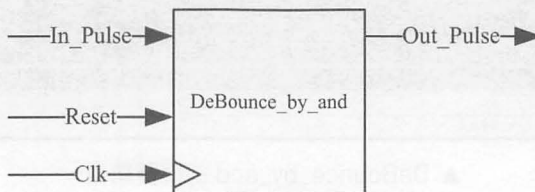
同樣，先前是邏輯‘0’接著至少有連續 2 個時脈訊號都看到邏輯‘1’，那麼就輸出 1 個時脈訊號寬度的邏輯‘1’。

除彈跳電路與單一脈波電路至少都可以用 and 閘對輸入 pulse 做分析，或者用有限狀態機 (FSM) 對輸入 pulse 做分析，這二種方式來設計。

11.5.1 用 and 閘對輸入 pulse 做分析 (DeBounce by and)

用 and 閘對輸入 pulse 做分析 (DeBounce by and)，當一開始是邏輯 0，接著至少有連續 2 個邏輯 1 時，才讓輸出的 pulse 等於邏輯 1，否則一定讓輸出的 pulse 等於邏輯 0。

```
Out_Pulse = In_Pulse_DL1 & In_Pulse_DL2 & (~In_Pulse_DL3)
```



▲ DeBounce_by_and 的電路方塊圖

【DeBounce_by_and 的 Verilog 程式碼：DeBounce_by_and.V】

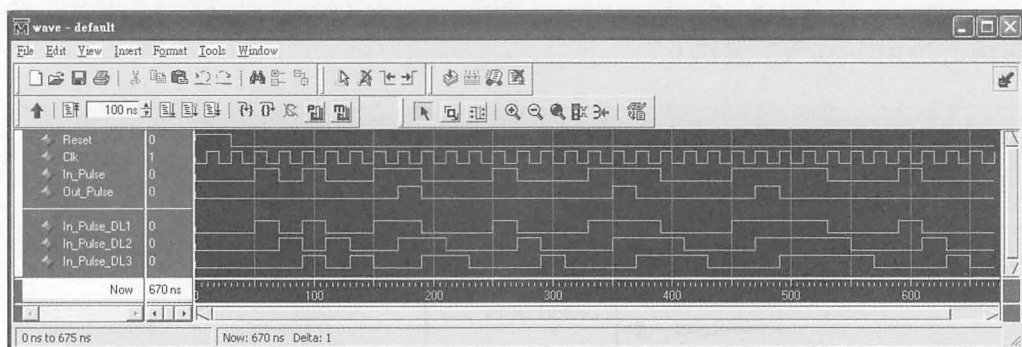
```
module DeBounce_by_and (Reset, Clk, In_Pulse, Out_Pulse);
input  Reset, Clk;
input  In_Pulse;
output Out_Pulse;

reg    In_Pulse_DL1, In_Pulse_DL2, In_Pulse_DL3;

always @(posedge Reset or posedge Clk)
begin // 正緣觸發
    if (Reset) // Reset = 1'b1
    begin
        In_Pulse_DL1 <= 1'b0;
        In_Pulse_DL2 <= 1'b0;
        In_Pulse_DL3 <= 1'b0;
    end
    else
    begin
        In_Pulse_DL1 <= In_Pulse;
        In_Pulse_DL2 <= In_Pulse_DL1;
        In_Pulse_DL3 <= In_Pulse_DL2;
    end
end
end
```

```
assign Out_Pulse = In_Pulse_DL1 & In_Pulse_DL2 & (~In_Pulse_DL3);

endmodule
```



▲ DeBounce_by_and 的模擬結果

DeBounce_by_and 模擬結果說明

- In_Pulse_DL1 是用 Clk 去對 In_Pulse 取樣的。
- In_Pulse_DL2 是用 Clk 去對 In_Pulse_DL1 取樣的。
- In_Pulse_DL3 是用 Clk 去對 In_Pulse_DL2 取樣的。
- In_Pulse_DL1 & In_Pulse_DL2 & (~In_Pulse_DL3) 這一行就是用 and 閘對輸入 pulse 做分析 (DeBounce by and)，當一開始是邏輯 0 (In_Pulse_DL3)，接著至少有連續 2 個 (In_Pulse_DL2 與 In_Pulse_DL1) 是邏輯 1 時才讓輸出的 pulse 等於邏輯 1，否則一定讓輸出的 pulse 等於邏輯 0。

11.5.2 用有限狀態機 (FSM) 對輸入 pulse 做分析 (DeBounce by FSM)

用有限狀態機 (FSM) 對輸入 pulse 做分析 (DeBounce by FSM)，當有連

續 2 個邏輯 1 時才讓輸出的 pulse 等於邏輯 1，否則一定讓輸出的 pulse 等於邏輯 0。



▲ DeBounce_by_FSM 的電路方塊圖

【DeBounce_by_FSM 的 Verilog 程式碼 DeBounce_by_FSM.V】

```
// 定義各個電路的狀態編碼
`define Detect_1st_1    2'h1
`define Detect_2nd_1    2'h2
`define Detect_0        2'h3

module DeBounce_by_FSM (Reset, Clk, In_Pulse, Out_Pulse);
input  Reset, Clk;
input  In_Pulse;
output Out_Pulse;

reg    Out_Pulse;

reg [1:0] State;

// 定義狀態的值
//parameter Detect_1st_1 = 2'h1, Detect_2nd_1 = 2'h2, Detect_0 = 2'h3;

always @(posedge Reset or posedge Clk)
begin // 正緣觸發
    if (Reset) // Reset = 1'b1
    begin
        Out_Pulse <= 1'b0;
        State <= `Detect_1st_1;
    end
    else
    begin
        Out_Pulse <= 1'b0; /* 預設輸出訊號 Out_Pulse 為 0 */

        /* 依據 State 及外界輸入的訊號，來決定 State 及輸出訊號 Out_Pulse */
        case (State)
```

```

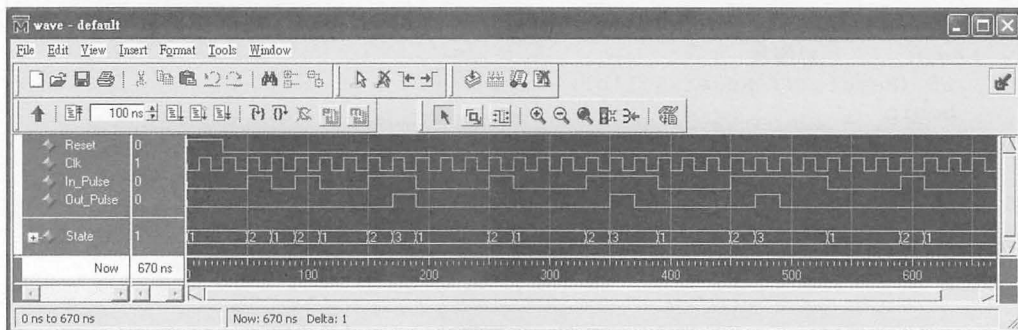
Detect_1st_1:
  if (In_Pulse)
    begin
      State <= `Detect_2nd_1;
    end

Detect_2nd_1:
  if (In_Pulse)
    begin
      Out_Pulse <= 1'b1;
      State <= `Detect_0;
    end
  else
    begin
      State <= `Detect_1st_1;
    end

Detect_0:
  if (!In_Pulse)
    begin
      State <= `Detect_1st_1;
    end

/* 若 State 不為有效的狀態，則將 State 設定為 Detect_1st_1 */
default:
  begin
    State <= `Detect_1st_1;
  end
endcase
end
end
endmodule

```



▲ DeBounce_by_FSM 的模擬結果

DeBounce_by_FSM 模擬結果說明

用有限狀態機 (FSM) 對輸入 pulse 做分析 (DeBounce by FSM)，當有連續 2 個邏輯 1 時 (State 從 Detect_1st_1/2'h1 到 Detect_2nd_1/2'h2) 才讓輸出的 pulse 等於邏輯 1，接著狀態並且再到 Detect_0/2'h3，要等到看到邏輯 0 時才會回到狀態 Detect_1st_1；否則一定讓輸出的 pulse 等於邏輯 0。



11.6 非同步 Reset

為了避免非同步的 Reset 和正緣觸發的 Clock 訊號同時到來，造成電路不正常的運作，您可以增加一個由 Clock 在負緣時對 Reset 取樣得到另一個取樣後的 sReset，使得 sReset 傳給後級一定是在 Clock 為負緣的時候變化，因而可以避免這個可能造成電路不正常運作的情形。

例如：Async_Reset.V

```
module Async_Reset (Reset, Clock, Data, Out);
input      Reset, Clock;
input      [15:0] Data;
output     [15:0] Out;
reg        [15:0] Out;

reg        sReset;

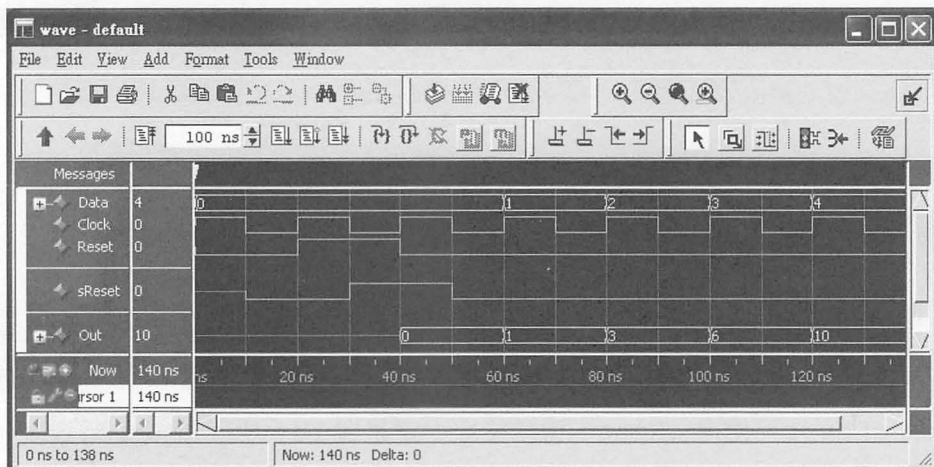
always @(negedge Clock)
begin
    sReset <= Reset; // Clock 在負緣時對 Reset 取樣
end

always @(posedge Clock)
begin
    if (sReset) // Clock 在負緣時對 Reset 取樣得到另一個取樣後的 sReset
```

```

        Out = 0;
    else
        Out = Out + Data;
    end
endmodule

```



▲ Async_Reset 的模擬結果



11.7 節省電力的基本方法

將目前用不到的電路關掉 (Disable 或 Bypass)，關掉方法通常是讓該電路的時脈訊號 Clock 不要動作，因此電路的循序邏輯電路與暫存器就不會有變化，甚至也會讓相關的組合邏輯電路不會動作，達到省電的目的；例如：系統中的某一塊電路在某些情況下不需要運作，那麼就可以將它暫時關閉。

另外也可以關閉不需要運作的記憶體 (SSRAM、SDRAM、DDR... 等等) 電路的時脈訊號 Clock，尤其是 DRAM 這一類的記憶體需要 Refresh 刷新/Pre-charge 預充電的動作是需要消耗系統電力的。

本章練習**Question 1 :**

請想想看還有沒有其他節省面積的電路設計實例？！

Question 2 :

請想想看還有沒有其他節省電力的電路設計實例？！

關掉(Disable 或 Bypass)不需要用到的電路，例如：某一 path 的電路在某些情況下不需運作，則可以將它暫時關閉。另外關閉不需要運作的記憶體(SSRAM、SDRAM、DDR、...等等)，尤其是 DRAM 這一類的記憶體需要 Pre-charge (預充電) 與 Refresh (刷新)。

Question 3 :

請想想看還有沒有其他加快電路運算速度的電路設計實例？！

Question 4 :

32 Bits 的加法器運算速度比較慢，可以改用 2 個 16 Bits 的加法器來做？！

PS Google 可以幫您找到解答。

